Streaming Videos

Placer des vidéos dans des caches pour réduire la latence

Google Hash Code 2017, problème de qualification

Mamadou Ougailou Diallo, Ibrahima Camara, Abdelhamid Ouazzou, Achour Djerada
M2 Problem Solving

Sommaire

1 **Algorithmique**

le problème, notre méthode, un exemple pas à pas

2 **Évaluation expérimentale**

protocole, résultats, comparaison de nos versions, SWOT

3 **Code**

l'organisation du projet et les choix d'implémentation

4 **Conclusion**

ce que nous retenons du travail

Questions

annexes disponibles

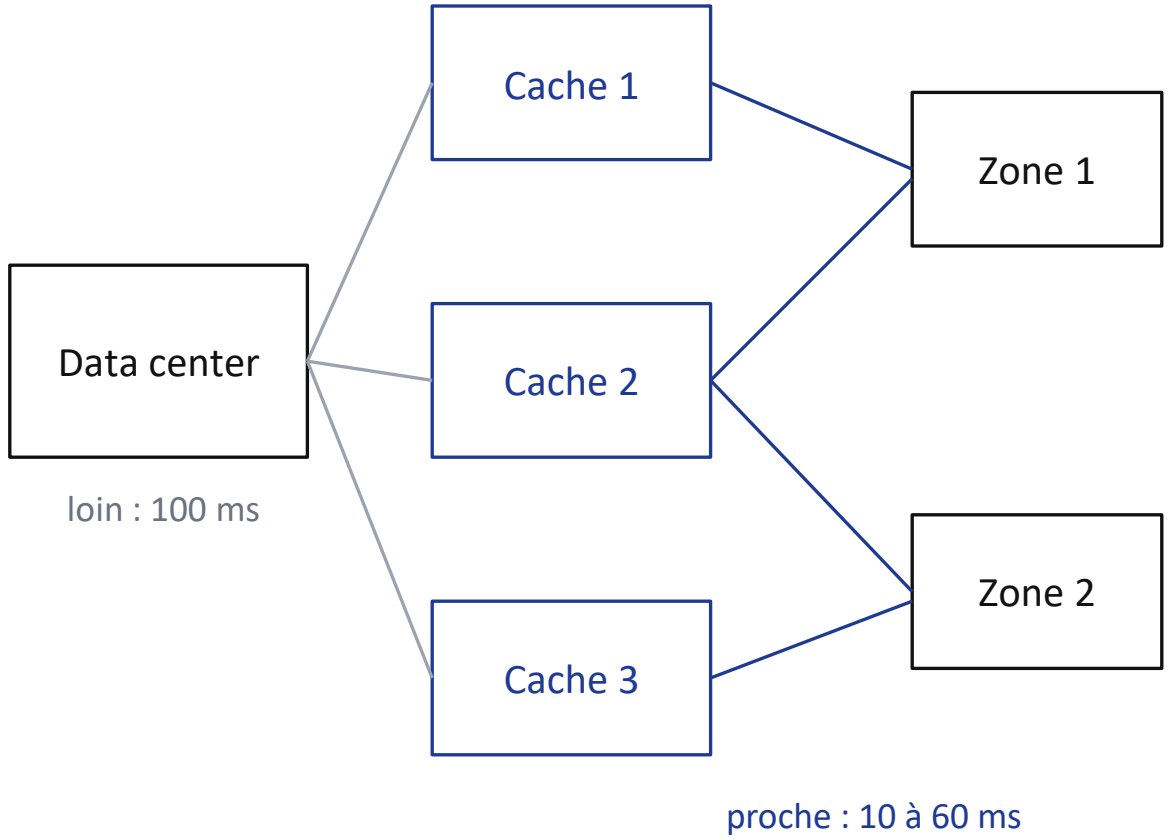
1

Algorithmique

Le problème, notre méthode, un exemple pas à pas

Le problème : rapprocher les vidéos des utilisateurs

- Un data center loin des utilisateurs contient toutes les vidéos.
- Des caches proches peuvent en garder des copies, mais leur capacité est limitée.
- Question : quelles vidéos mettre dans quels caches pour réduire le plus possible le temps d'attente ?



Score = 1000 × (ms gagnées, sommées sur toutes les requêtes) / (nombre de requêtes)

Pourquoi c'est difficile

10 000

vidéos

998 964

lignes de requêtes

20 × 2 000 Mo

caches

Plus grande instance de nos tests (instance 14)

- Remplir un seul cache est déjà un problème du sac à dos.
- Les caches dépendent les uns des autres : une vidéo déjà présente ailleurs rapporte moins.
- Dans 11 instances sur 19, les vidéos demandées ne tiennent pas toutes dans les caches : il faut choisir.

Notre méthode en trois idées

1

Placer d'abord ce qui rapporte le plus

On classe les couples (cache, vidéo) par gain par Mo, on place le meilleur, puis on recalcule les gains.

2

Refaire chaque cache au mieux

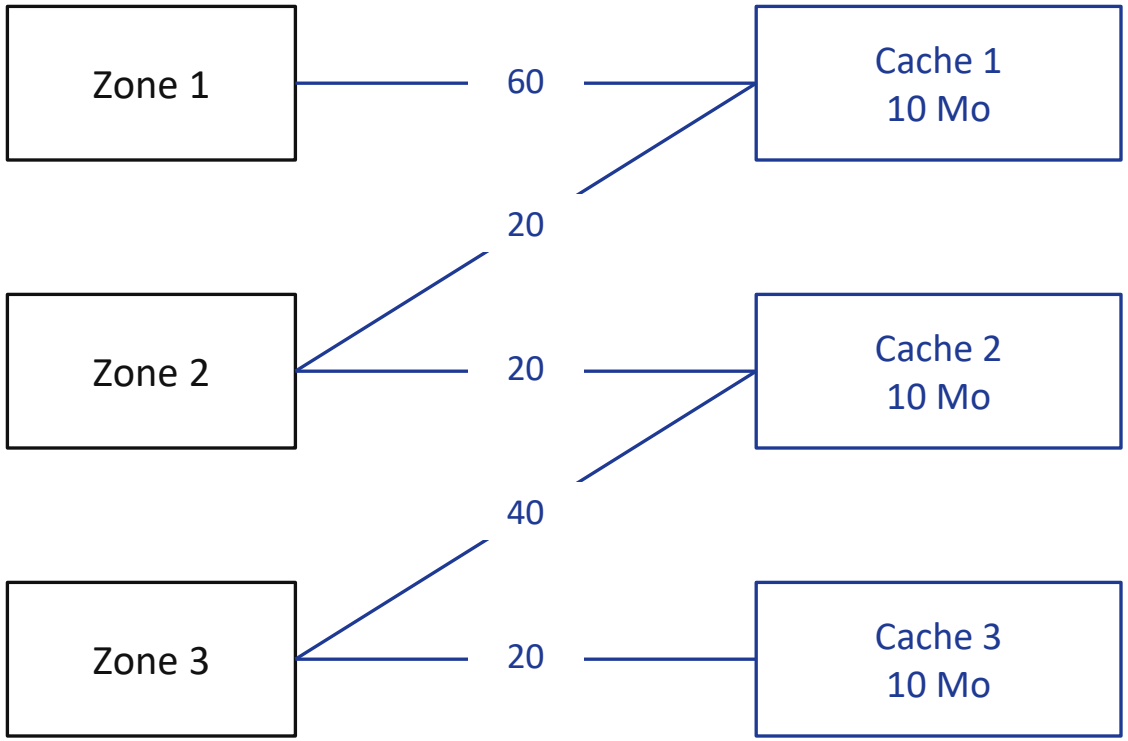
Les autres caches restent fixes : on choisit exactement les vidéos qui maximisent le gain dans la capacité.

3

Secouer et réessayer

On vide quelques caches voisins, on les remplit à nouveau, et on garde le résultat seulement s'il est meilleur.

Exemple : une instance de cinq vidéos



Chaque zone atteint le data center en 100 ms (latence en ms sur les traits).

Vidéo (taille)	Zone	Requêtes
A (4 Mo)	Zone 2	60
B (3 Mo)	Zone 1	40
C (4 Mo)	Zone 3	30
C (4 Mo)	Zone 1	50
D (7 Mo)	Zone 3	50

La vidéo E (7 Mo) n'est demandée par personne.

Idée 1 : placer le plus rentable, puis recalculer

gain par Mo (ms) (ms)	A	B	C	D
Cache 1	**1200**	533	500	-
Cache 2	1200	-	450	429
Cache 3	-	-	600	571

Gain par Mo = ms gagnées sur toutes les requêtes servies, divisées par la taille de la vidéo.

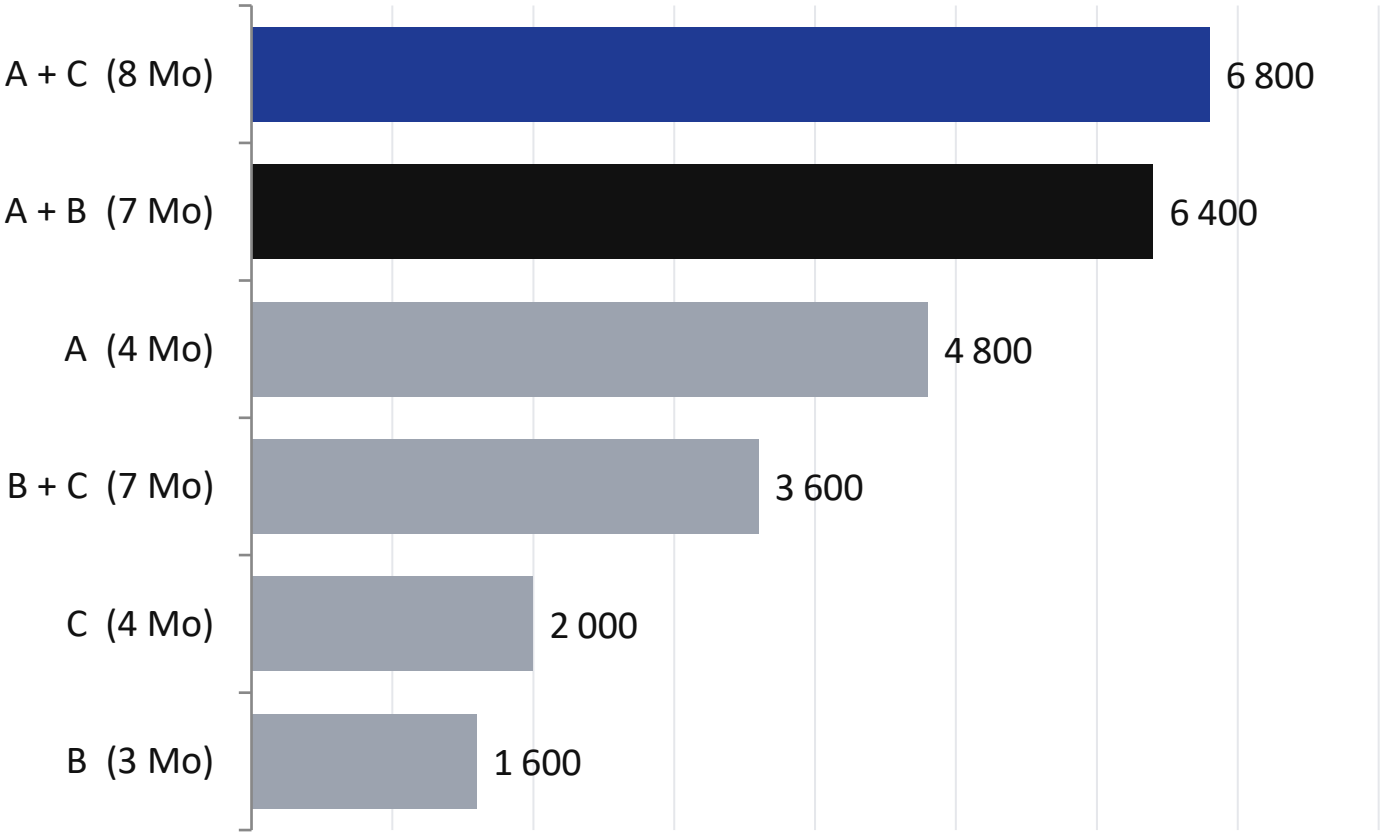
1. A dans le cache 1 (1200 par Mo).

   A dans le cache 2 tombe à 0 : la zone 2 est déjà servie à 20 ms.

2. C dans le cache 3 (600 par Mo).
3. D ne rentre plus dans le cache 3 (4 + 7 > 10). B dans le cache 1 (533).
4. D dans le cache 2 (429).

Résultat : cache 1 = {A, B}, cache 2 = {D}, cache 3 = {C}
Gain moyen : **51,3 ms par requête**

Idée 2 : refaire chaque cache exactement



Cache 1, les autres caches ne bougent pas. Les combinaisons à 11 Mo (A + B + C) ne rentrent C) ne rentrent pas.

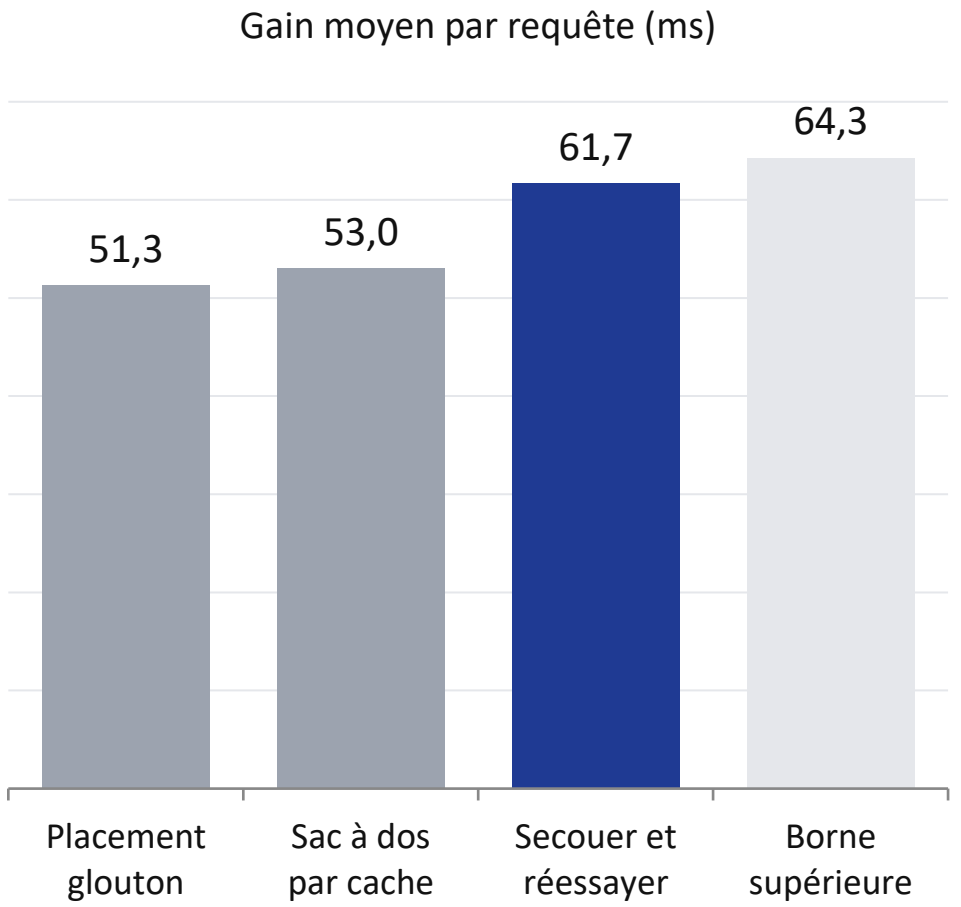
- Le glouton avait pris A + B (en noir) car B a un meilleur gain par Mo que C.
- A + C (en bleu) gagne 400 de plus : C remplit mieux les 10 Mo.
- Un sac à dos exact teste intelligemment toutes les combinaisons.

Gain moyen : **53,0 ms par requête**

Idée 3 : secouer et réessayer

	Cache 1	Cache 2	Cache 3
Avant	A , C	D	C
Après	B, C	A , C	D

- On vide trois caches voisins : ils partagent des zones.
- On les remplit à nouveau, chacun au mieux.
- Si le score baisse, on annule. Sinon on garde.
- Avant, C était dupliquée dans les caches 1 et 3.



Combien ça coûte ?

Étape	Ce qu'on fait	Ce que ça coûte
Lire le fichier	On lit chaque nombre une fois.	Proportionnel à la taille du fichier.
Placement glouton	Chaque requête est examinée une fois par cache qu'elle voit. Ensuite, on ne recalcule que le meilleur candidat.	Proportionnel à : requêtes × caches vus par zone.
Sac à dos par cache	On trie les vidéos intéressantes et on ne calcule exactement que ce qui reste incertain.	Au pire : vidéos × capacité du cache. En pratique, bien moins.
Secouer et réessayer	Chaque essai ne touche que quelques caches.	Ne dépend pas du nombre total de caches. Le temps disponible fixe le nombre d'essais.

Aucune étape n'essaie toutes les combinaisons possibles : c'est ce qui rend la méthode utilisable sur un million de un million de requêtes.

2

Évaluation expérimentale

Protocole, résultats, comparaison de nos versions, SWOT

Protocole expérimental

Instances	4 officielles (Hash Code 2017) + 15 du dossier instances_k
Tailles	de 100 à 10 000 vidéos, de 100 à 1 000 000 lignes de requêtes
Machine	Intel Core i5-8365U (4 cœurs, 8 threads), 16 Go de RAM, Java 24, tas de 6 Go
Réglages	20 s par instance et 6 threads (budgets et threads variés plus loin), graines fixes
Mesure	Score du juge officiel ; rapport à la borne supérieure ; gain de chaque étape
Répétitions	1 exécution par instance, 3 sur zoo pour la variabilité ; 2 pour les threads

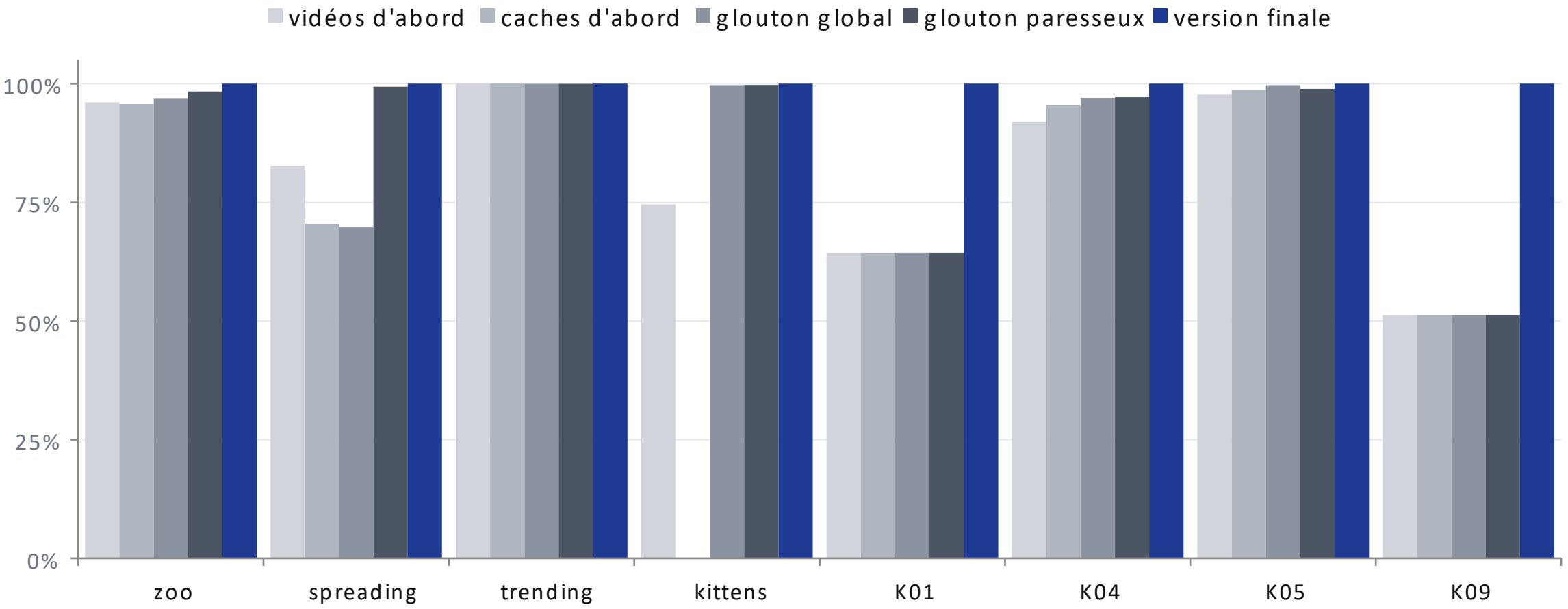
Limites. La borne ignore la capacité des caches : elle est lâche quand les caches sont saturés. Les temps réels et les temps réels et les threads rendent deux exécutions légèrement différentes. Une seule machine, portable. portable.

11 instances sur 19 ont plus de vidéos que de place



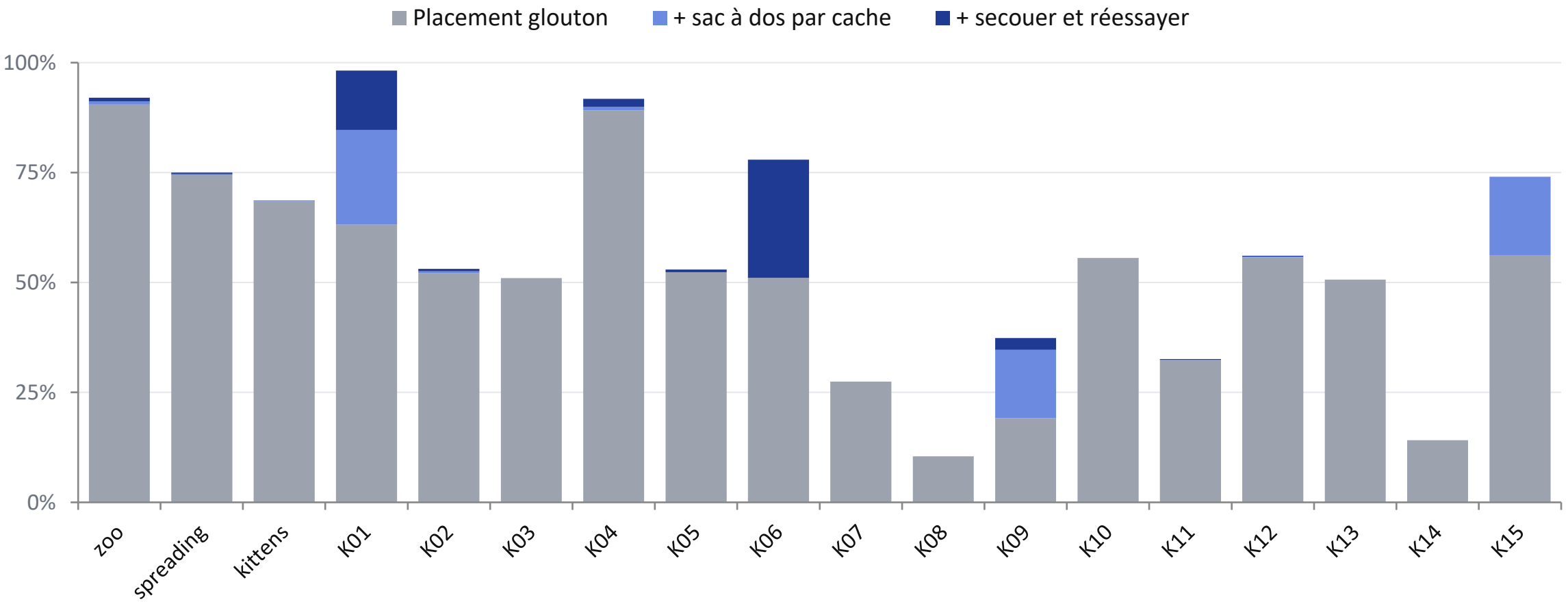
Pression = taille des vidéos demandées / capacité totale des caches (échelle logarithmique). 1 = juste assez de place. Bleu : instances officielles, gris : instances K.

Le score de nos cinq versions de solveur



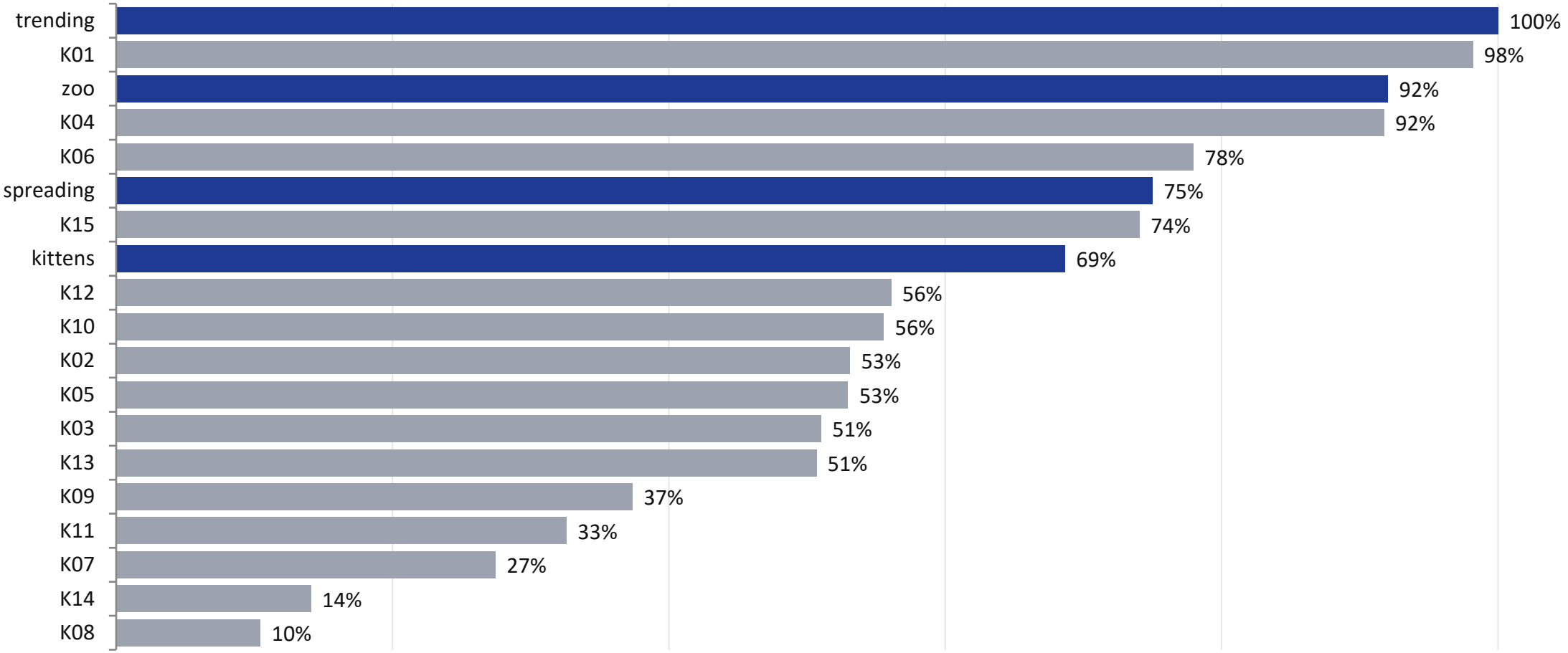
Score du juge officiel, rapporté à celui de la version finale (100 %). Les budgets : version finale 20 s et 6 threads, autres versions sans limite (300 s maximum). Barre absente : exécution au-delà de 300 s.

Ce que chaque étape apporte, instance par instance



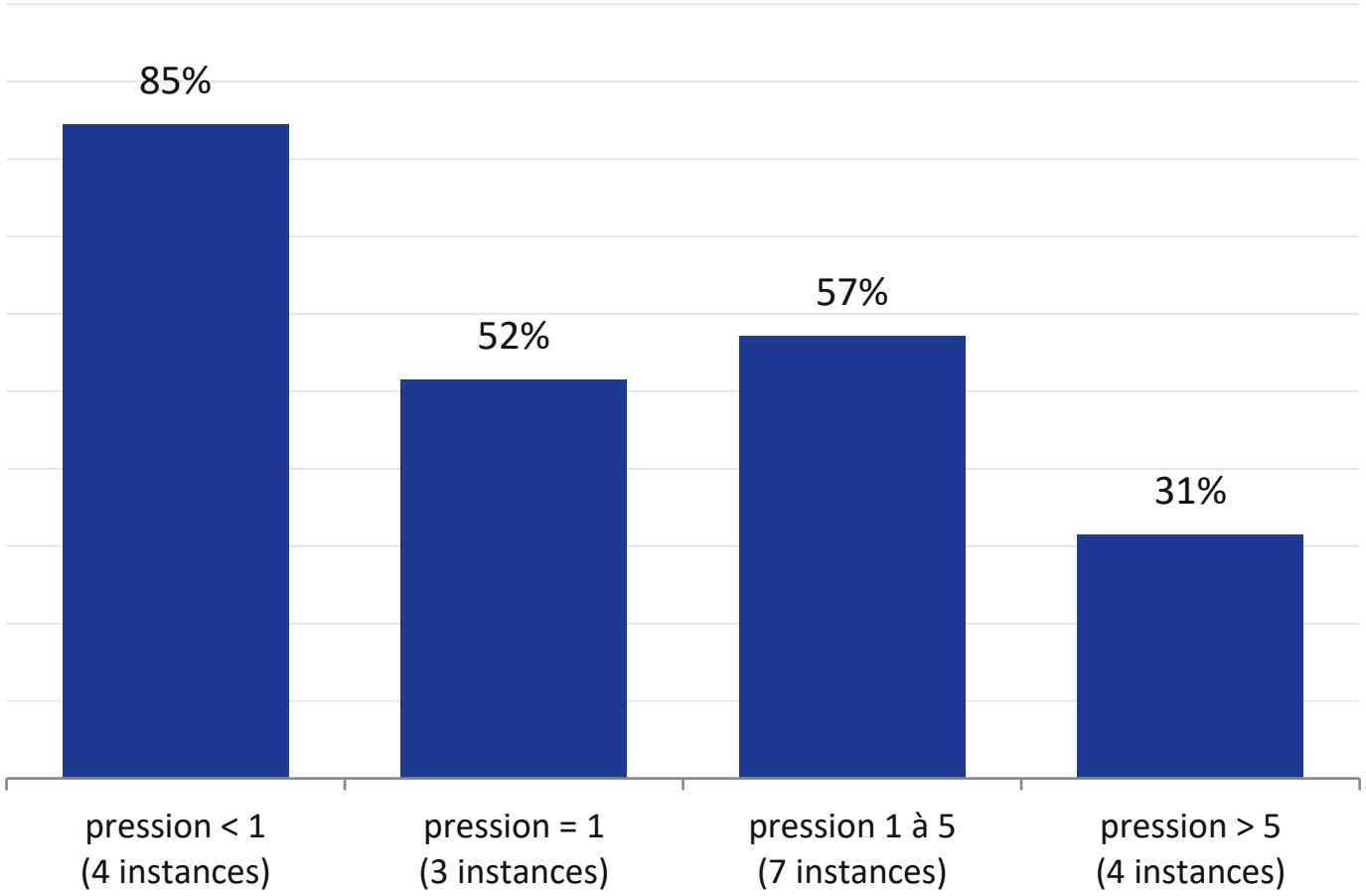
Part de la borne supérieure atteinte. trending_today (résolue exactement, 100 %) n'est pas représentée.

Part de la borne supérieure atteinte : de 10 % à 100 %



Score final / borne supérieure. Bleu : instances officielles, gris : instances K.

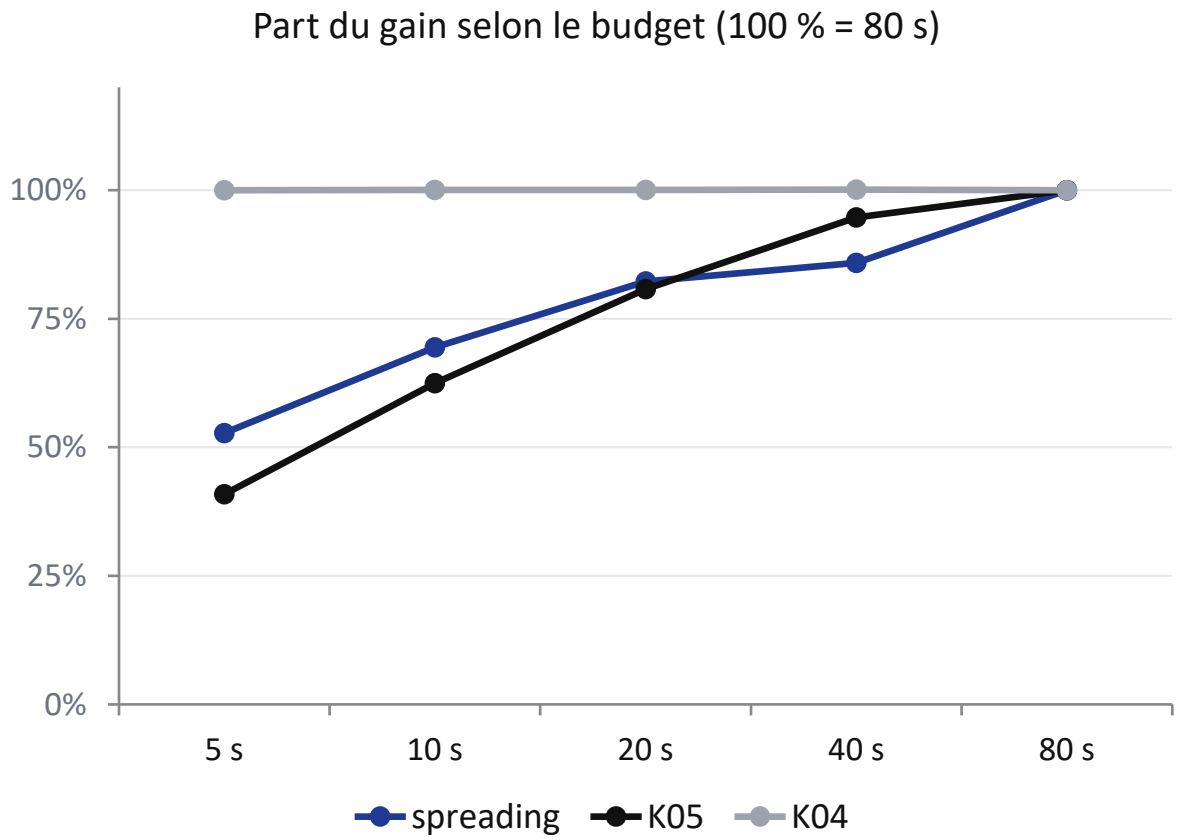
Les instances très saturées sont les plus loin de la borne

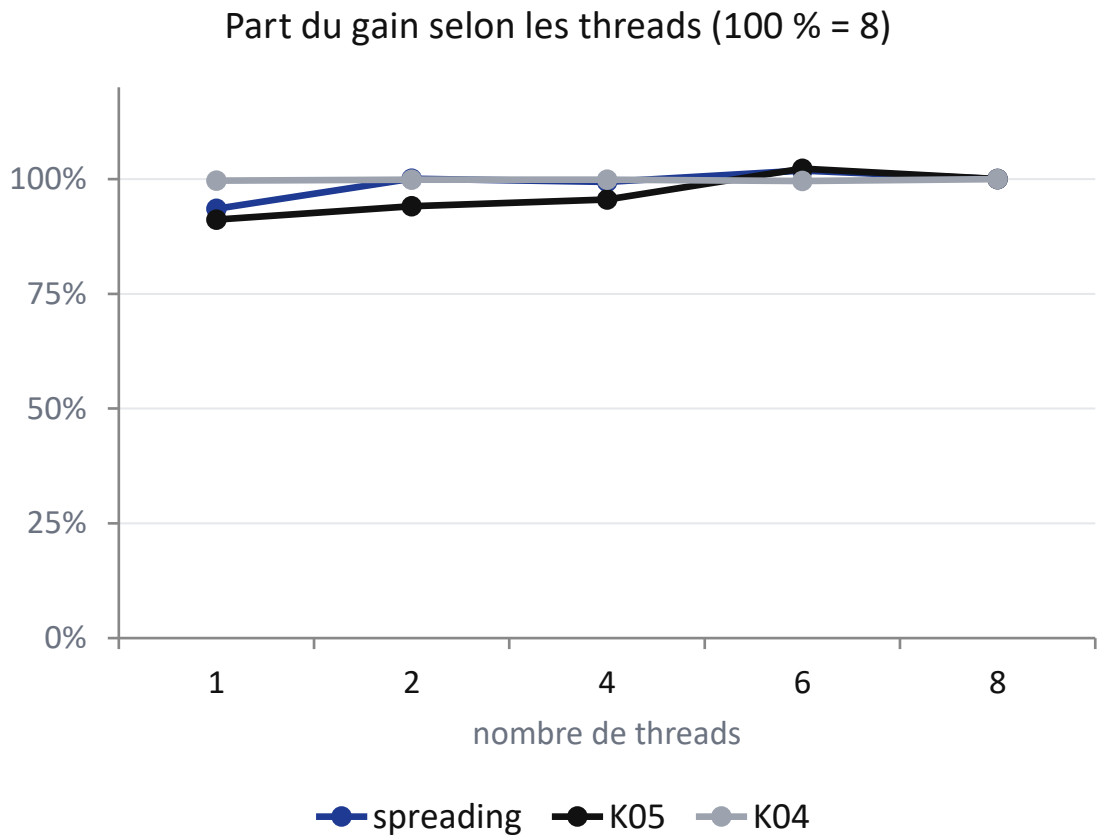


Part moyenne de la borne selon la pression de capacité (trending_today exclue).

- La borne sert chaque requête avec son son meilleur cache, sans limite de place.
- Quand les caches sont très pleins (pression > 5), cette hypothèse est très optimiste : 31 % contre 85 %.
- Corrélation entre la pression (en log) et la part de la borne : -0,56.
- Un faible pourcentage ne signifie donc pas forcément une mauvaise solution.

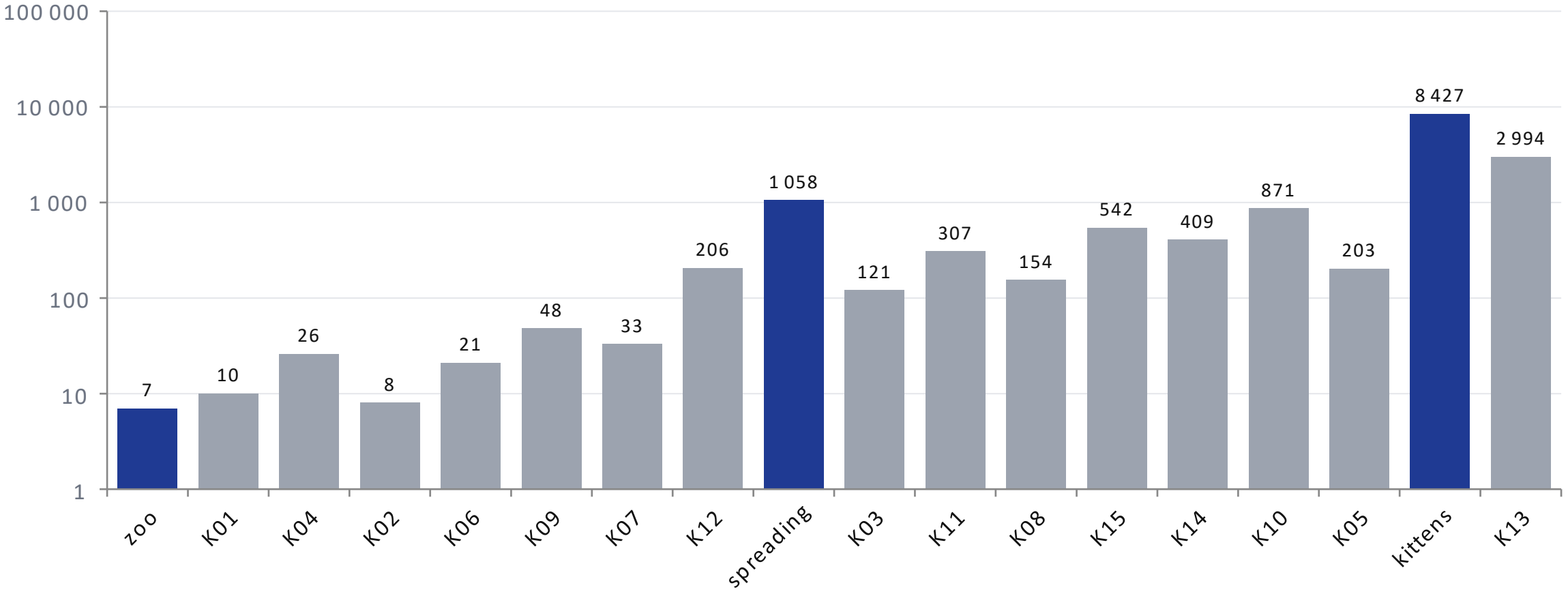
Plus de temps aide parfois, plus de threads presque jamais





Gain = score moins score du placement glouton. Gauche : 6 threads, 1 exécution par point. Droite : budget de 30 s, 2 exécutions par point ; la machine a 4 cœurs physiques.

Le temps mesuré augmente avec le travail prévu



Temps du placement glouton en ms (échelle logarithmique), instances triées par travail prévu croissant (requêtes × caches vus par zone), de 337 à 100 000 000. Corrélation 0,92. Bleu : officielles, gris : instances K.

SWOT

Forces

- Chaque cache est optimisé exactement.
- Solutions toujours valides : vérifiées par le juge officiel.
- Jusqu'à +95 % sur le placement glouton.
- Optimum prouvé sur trending.

Faiblesses

- La borne est lâche : on ne connaît pas la distance à l'optimum.
- Aucun gain après le glouton sur 3 instances (K03, K07, K13).
- Résultats un peu variables d'une exécution à l'autre.

Opportunités

- Une borne qui tient compte de la capacité.
- Voisinages plus riches pour l'étape 3.
- Plus de cœurs : la recherche se parallélise bien.

Menaces

- Instances plus grandes : mémoire (caches × vidéos) et temps.
- Dépendance au budget de temps et à la machine.
- Jeu d'instances limité à 19 cas.

3

Code

L'organisation du projet et les choix d'implémentation

Un projet Java modulaire, un solveur interchangeable

Le programme

model	Video, Endpoint, Cache, RequestDescription, Problem
io	lecture rapide du fichier, écriture de la solution
solver	une interface, cinq solveurs interchangeables
solveur final	problème à plat, suivi des latences, placement glouton, sac à dos, perturbations, rangement exact

Autour du programme

- Juge officiel en C++, branché sur le Makefile.
- Makefile : compiler, lancer, juger en une commande.
- Pipeline Python : générer, valider, compiler, résoudre, juger.
- Générateur et validateur d'instances.
- Tableau de bord pour visualiser les instances et les solutions.

Travail en équipe : une branche par fonctionnalité, 5 pull requests fusionnées, du 14 septembre au 19 octobre.

Solveur final : les choix d'implémentation



- Tableaux d'entiers à plat plutôt que des objets : moins de mémoire, plus de cache CPU.
- Une solution = un tableau de bits (cache, vidéo).
- Meilleure et deuxième meilleure latence par requête : l'effet d'un ajout ou d'un retrait se calcule en temps constant.
- Threads : chacun explore, un maître fusionne les améliorations validées par le score.
- À chaque étape : capacité respectée, score incrémental = score recalculé, score jamais en baisse.

```
private void offerLatency(int request,
                          int latency, int cache) {
  if (latency < bestLatency[request]) {
    secondBestLatency[request] = bestLatency[request];
    bestLatency[request] = latency;
    bestCache[request] = cache;
  } else if (latency < secondBestLatency[request]) {
    secondBestLatency[request] = latency;
  }
}
```

Extrait de notre code (suivi des latences).

4

Conclusion

Ce que nous retenons du travail

Conclusion

5

versions de solveur, en cinq semaines de projet

+17 %

de score entre « vidéos d'abord » et la version finale (instances officielles)

4

étudiants, un juge officiel, des outils outils pour comparer

Ce que nous retenons

- Recalculer les gains après chaque placement rapporte plus qu'un ordre fixé au départ.
- Résoudre chaque cache exactement, puis perturber de petits groupes de caches, apporte les derniers points.
- Un juge officiel et nos outils nous ont permis de comparer nos versions sur les mêmes instances.

Pour aller plus loin : une borne qui tient compte de la capacité, des perturbations mieux choisies.